# Unidad 4 - POO: Métodos, Funciones y Clases (Versión Reducida)

### Introducción a POO

La Programación Orientada a Objetos (POO) usa **clases** como plantillas para crear **objetos**. Cada objeto tiene **atributos** (datos) y **métodos** (funciones).

- **Métodos estáticos**: Pertenecen a la clase, no a objetos. Se llaman sin crear instancia: `Clase.metodo()`.
- **Métodos de instancia**: Pertenecen a objetos específicos.

Ejemplo básico:

```
public class Ejemplo {
    static int contador = 0;  // Variable de clase
    int id;  // Variable de instancia
    public static void main(String[] args) {  // Método estático
        Ejemplo obj = new Ejemplo();
        obj.mostrarId();
    }
    public void mostrarId() {  // Método de instancia
        int temp = 5;  // Variable temporal
        System.out.println("ID: " + id);
    }
}
```

### Variables en Clases

- **Variables de Clase** (`static`): Compartidas por todas las instancias.
- **Variables de Instancia**: Únicas por objeto.
- **Variables Temporales**: Solo existen dentro de un método.

### Definición de Métodos

Sintaxis: `modificadores static tipo_retorno nombre(parámetros) { ... return valor; }`

- **Modificadores**: `public`, `private`, `protected`, `static`.
- **Tipo retorno**: `void` (nada), `int`, `String`, etc.
- **Parámetros**: Lista de variables de entrada.

Ejemplos:

```
public static int sumar(int a, int b) {
    return a + b;
}
private void mostrarMensaje(String msg) {
    System.out.println(msg);
}
```

### Sobrecarga de Métodos

Múltiples métodos con mismo nombre pero diferentes parámetros (número o tipo).

```
public int sumar(int a, int b) { return a + b; }
public int sumar(int a, int b, int c) { return a + b + c; }
public double sumar(double a, double b) { return a + b; }
```

### Paquetes y Organización

- **Paquetes**: Agrupan clases relacionadas. Declaración: `package nombre;`
- **Import**: `import paquete.Clase;` o `import paquete.*;`

Ejemplo:

```
// En calculos/Matematicas.java
package calculos;
public class Matematicas {
    public static int doble(int n) { return n * 2; }
}
// En main/Programa.java
package main;
import calculos.Matematicas;
public class Programa {
```

```
    public static void main(String[] args) {
        int resultado = Matematicas.doble(5);
    }
}
```

### Llamadas entre Métodos

- **Mismo objeto**: `this.metodo()`
- **Otra clase**: `OtraClase.metodo()` (si estático)
- **Instancia**: `objeto.metodo()`

Ejemplo:

```
public class Calculadora {
    public static int sumar(int a, int b) {
        return a + b;
    }
    public int multiplicar(int a, int b) {
        return a * b;
    }
    public void calcular() {
        int suma = sumar(2, 3);  // Llama estático
        int mult = this.multiplicar(4, 5);  // Llama instancia
    }
}
```

### Excepciones Básicas

Manejan errores. Lanzar: `throw new TipoExcepcion("mensaje");`

Tipos comunes: `IllegalArgumentException`, `ArithmeticException`.

Ejemplo:

```
public static int dividir(int a, int b) {
    if (b == 0) {
        throw new IllegalArgumentException("Divisor no puede ser cero");
    }
    return a / b;
}
// Uso con try-catch
try {
    int resultado = dividir(10, 0);
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
```

### Librerías Estándar Útiles

- **Scanner**: Entrada de usuario. `import java.util.Scanner;`
- **Math**: Funciones matemáticas. `Math.abs(x)`, `Math.random()`
- **Arrays**: Manipulación de arrays. `Arrays.sort(arr)`, `Arrays.toString(arr)`

Ejemplo:

```
import java.util.Scanner;
import java.util.Arrays;
public class EjemploLibrerias {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] nums = {3, 1, 4, 1, 5};
        Arrays.sort(nums);
        System.out.println(Arrays.toString(nums));
        sc.close();
    }
}
```